# 类型化判断与确定性执行：面向消息通道的同意门控智能体协作网关架构

**蔡亦张（Yizhang Cai）**

北京邮电大学人工智能学院，中国北京

**摘要**

消息通道把用户接入智能体网络，同时也把非结构化、可能重放的事件，接到了经过认证、带有状态的协议调用上。本文提出一种将类型化判断与确定性执行分离的同意门控网关架构。消息到智能体的路径由七个模块组成：入站认证、身份与会话隔离、能力发现与路由、同意与策略控制、身份绑定协议执行、状态与幂等、投递与可观测性。授权、身份、隔离和幂等四条不变式始终由代码保证。类型化判断层围绕意图、路由、歧义、同意、重放、重试和注入回答七个封闭问题，只返回封闭标签或概率，既不生成自由文本，也不产生工具调用，并且只能为路由、同意与恢复策略提供阈值参数。因此，认证、端点身份、协议状态和外部副作用始终处于模型控制之外。我们将架构映射到 iMessage/Photon 入站与 Agent Collaboration Protocols（ACPs）/Agent Interaction Protocol（AIP）v2.2 Leader-Partner 路径，实测其中的发现与推荐环节，并记录一次经同意门控放行的调用及其在伙伴侧产生的副作用。由固定模型生成的 130 条请求经真实入站投递三次：网关在 50 条 direct 请求中分别推荐目标受测 Partner 48、48、50 条；当请求同时符合两个能力时推荐多于一个智能体；当没有任何受测 Partner 宣传相应能力时不推荐任何受测 Partner。配对消融显示：sender-to-peer 断言拒绝身份不一致（0/30 对 30/30）；调用者/会话分区阻止跨调用者读取（0/30 对 30/30）；事件级幂等存储在重复投递时把处理次数减半；在已部署适配器上，用户同意前没有任何 Partner 动作被记录（0/40），每次同意恰好产生一条动作记录（36/36）。另有一项留出研究，在一个已部署模型上重复运行三次，比较类型化与自由文本两种答案形态：类型化在经人工复核的标签上更准确、校准更好；在专门的注入压力集上两种形态都能识别注入，却都会把约一半的引用型文本误判为引导尝试。以上结果来自一个部署与一套能力集；网关自身的开销另行分项测量。本文因此划出一条明确的架构与评测边界：模型只能为策略提供建议，不能授权任何外部副作用。

**关键词：** 多智能体系统；智能体互联；消息网关；同意控制；身份绑定；类型化决策模型；校准；AIP；ACPs；可靠性；安全性评测

## 1 引言

大模型智能体一方面越来越多地经由消息通道触达用户，另一方面越来越多地通过结构化协议与其他智能体相连。这两类接口的假设并不相同：消息通道标识的是一段人与人的会话，承载的是非结构化、可能重复或延迟的事件；而智能体间协议要求经过认证的端点、结构化的请求、显式的任务状态和机器可验证的身份。因此，把两者接起来的网关必须先完成一组有限的判断，才可能发起任何调用：这条消息是否要求执行任务、应该交给哪个智能体、歧义是否大到必须请用户澄清、操作是否带有需要用户同意的副作用、事件是否为语义重放、失败能否重试，以及入站内容是否在试图操纵网关。

现有两种做法都不理想。让自由生成文本的大模型来做判断，会带来成本、时延与不受约束的输出空间：模型可以输出任意文本或工具调用，于是一次错误判断就可能被当成越权调用执行。反过来，把判断写成静态规则虽然避免了这一风险，却处理不了自然语言中的歧义，网关就只能在两种坏结果之间选择：要么频繁打断用户，要么按浅层匹配选错目标。

本文研究第三条路线：把判断与执行分开。类型化决策模型只回答范围很窄的问题，并且只返回带概率的标签——针对是非命题的二元概率、针对一组命名选项的概率分布，或者一个有序评分标尺上的位置。它不生成自由文本，也不产生工具调用，所有阈值、回退与副作用都由网关的策略代码负责。我们把这种分离实现为一个七模块的网关架构，并在 iMessage/Photon 入站路径与 ACPs/AIP v2.2 执行路径上对它展开研究。

本文的贡献如下：

1. 一个把类型化判断与确定性执行分离的网关架构，由七个模块构成、受四条不变式约束。判断层只能为发现与路由、同意控制、状态管理三个模块的参数化阈值提供输入；入站认证、身份校验与协议执行仍由密码学与确定性代码实现。
2. 一份七个判断点的清单，每个判断点都有答案形态、归属模块与声明的失败后果，把原本停留在设计讨论中的取舍变成可逐点测量的对象。
3. 一套面向判断层的校准感知评测协议（Brier 分数、校准误差、覆盖—风险曲线、注入条件下的错误）；以及一套部署级协议，用于在真实请求上测量路由，并测量各项由环境变量控制、各带同代码对照的不变式消融。
4. 一套可复现产物：模型生成的请求套件、逐条路由结果、带同代码对照的环境变量消融构建、已注册 Partner 的配置、可在别处重复该协议的实验装置，以及顾问模式下的判断日志，见 `https://github.com/ankelas/consent-gated-gateway-artifact`。

本文的证据分三类：架构、七个判断点与状态机属于设计；实机测量覆盖已部署入站上的发现与推荐、三项不变式消融、在已部署适配器上完成的同意门控与端到端执行（伙伴侧的副作用有记录），以及同一部署在真实事件上以顾问模式咨询判断平面；判断层则在独立套件上单独测量，标签先由构造生成，再经独立复读与仲裁，最后由人工复核。所有结果都来自一个部署、一个注册中心与一套能力集。

## 2 相关工作

### 2.1 评测与消息入口协议

现有多智能体评测往往只强调最终任务准确率或单一应用流程，这可以衡量能力，却无法分离发现、授权、身份、传输、状态与投递各自的作用。评测套件 MAESTRO [16] 针对这一缺口，把异构多智能体系统接入统一运行时，可重复地执行任务、采集与框架无关的追踪，并比较时延、词元消耗、资源占用、消息体积、失败模式与调用图稳定性。另一项工作《Towards a Science of Scaling Agent Systems》[13] 在固定提示、工具与算力预算的前提下，比较单智能体、独立、集中式、去中心化与混合式协同结构，并把成功率与通信量、串行深度、冗余度、错误传播和任务结构联系起来。

本文沿用同样的受控对照原则，但改变了分析单位。研究对象是消息与智能体之间的网关：它用若干模块实现同意、身份绑定、会话隔离与幂等，而其中的判断点可以逐点测量。

本文原型沿用受检部署所用的 Agent Collaboration Protocols（ACPs）协议栈：Agent Interaction Protocol（AIP）v2.2 通过 `/rpc` 定义任务生命周期，Agent Identity Code（AIC）标识智能体，Agent Capability Specification（ACS）描述所声明的能力。已核对的仓库把规范标记为 `ACPs-spec-v02.02`，实现使用 SDK `v2.2.0`，固定提交为 `896e4bc79b95f11589e92eab06322430e5c64505` [1]。架构层面的问题不依赖这些细节：面向用户的消息传输并不是智能体间协议的调用方，因此网关必须完成两者之间的转换，而判断点就落在这道转换上。智能体互操作目前仍在标准化进程中，而不是已经定型：一项近期综述比较了 Model Context Protocol、Agent Communication Protocol、Agent-to-Agent 与 Agent Network Protocol，指出它们范围重叠，但身份与发现模型互不兼容 [6]。因此本文把协议层视为若干可替换的后端之一。

Li 等人提出 Agent Collaboration Protocols，把它作为面向智能体互联网（Internet of Agents）的体系化框架，给出智能体接入（onboarding）、探查（reconnaissance）、协作执行（engagement）与可观测（instrumentation）的方法，并指出现有 MCP、A2A、ANP 等智能体通信协议彼此割裂、只面向窄域应用 [14]。该框架以开源项目的形式落地，由北京邮电大学牵头、中国电子技术标准化研究院支持，参考实现包括协议规范、SDK、命令行工具、CA 认证服务、注册服务、发现服务、消息队列鉴权服务，以及 Leader 与 Partner 示例 [1]。本文的网关实现就建立在这套开源代码上：受检的 Leader 与 Partner 进程运行在 ACPs SDK 与相应基础设施之上，本文考察的身份码、能力描述、发现、任务生命周期与双向 TLS 等行为，都是 ACPs 本身的行为。

## 2.2 受限输出空间

约束模型的输出空间，可以让它的答案变得可检查。语法约束解码把生成限制在形式语言之内，无需微调即可改进结构化任务 [8]；而限制输出格式的提示，相对自由文本也会改变准确率 [20]。只返回封闭标签、命名选项上的分布或有序评分的决策模型，输出空间更小：一次回答不可能变成一次工具调用，阈值、回退与副作用都留在应用代码之中。

两类文献解释了策略层为什么需要拒答与校准。选择性分类的工作形式化了覆盖率与风险之间的权衡，并说明预测器在置信度低时应当能够拒绝作答 [7]；现代神经网络即便准确率很高，也常常校准不良，因此置信度必须被测量而不能被假定 [10]。对抗情形同样重要：提示注入可以改变 LLM 集成应用的工具调用 [9]，间接注入把指令藏在应用检索到的内容里 [15]，而智能体基准显示现有防御仍留有可观的残余风险 [4]。这些结果共同支持一种做法：网关测量选择性风险，同时把授权留在模型之外。

## 2.3 运行时约束及其边界

智能体安全的工作重心已经从「检测」转向「结构性约束」。InjecAgent [24] 与 ToolEmu [17] 这类基准显示，使用工具的智能体会被自己检索到的内容诱导去做有害动作；StruQ [3] 与 CaMeL [5] 的应对方式是控制「不受信任文本到动作空间」这条通路，而不是去判断文本本身是否恶意。另一条路线把约束放进运行时：Progent [19] 给单次工具调用附加权限约束，AgentSpec [21] 与 GuardAgent [23] 负责编译并检查运行时规则，IsolateGPT [22] 则把每个智能体的执行隔离开。这些系统都是在智能体内部施加约束；本文的场景不同：检查落在人类消息通道与协议之间的边界上，发起方是人，第一个问题就是这条请求究竟该不该发出去。这条边界也承接了更早的机制：委托授权已有成熟原语 [11]，而选择性预测给出了置信度不足时拒答的

原则性做法 [2, 7]。

# 3 问题定义与设计目标

设一条消息事件为 $e = (u, c, m, t)$，其中 $u$ 是调用者身份，$c$ 是会话标识，$m$ 是消息载荷，$t$ 是事件标识。设 $D$ 为可发现智能体集合，每个智能体带有能力描述与一个或多个协议端点。网关把事件映射为可选的协议任务：

$$G(e) \to (a, q, r) \tag{1}$$

其中 $a$ 是选中的智能体（网关弃答或请求澄清时为 $\emptyset$），$q$ 是结构化任务请求，$r$ 是面向用户的结果。

网关必须满足四条安全与正确性不变式：

- **授权不变式：** 除非入站认证通过、且所有策略要求的用户确认都已满足，否则不得发起任何外部智能体调用。确认适用于歧义路由，以及对外可见或被策略归类的副作用；纯读取操作不必一律确认。
- **身份不变式：** 被选端点的密码学身份必须符合所声明的智能体身份与能力策略。
- **隔离不变式：** 由 $(u, c)$ 派生的状态不得被其他调用者或会话读写。
- **幂等不变式：** 对给定幂等键，一个逻辑任务最多产生一次外部副作用，即使入站事件被重放。前两条来自经典的完全中介（complete mediation）与最小权限原则 [18]；最后一条是分布式请求处理中的幂等要求 [12]。

设计目标是：委派正确、失效关闭（fail-closed）、可恢复、协议开销有界，以及足以支持因果归因的遥测。

## 3.1 判断点

不变式规定的是「绝不允许发生什么」，并没有规定网关该怎么决策，而后者正是决策模型可以发挥作用的地方。表 1 列出本架构暴露的判断点：每一个都是一道答案空间封闭的问题，并有明确的归属模块与失败后果。

# 4 模块化架构

图 1 给出七个模块与判断平面。每个模块都由它要阻止的那一类失败来定义：

- **M1（入站）。** 校验 iMessage/Photon webhook、验证事件结构并归一化为内部事件；签名非法或格式错误的事件在发现之前就被终止。
- **M2（隔离）。** 由调用者与会话标识派生出稳定上下文，按该键对任务状态与回复做分区，并对进入遥测的标识做哈希或假名化。

表 1: 网关暴露的判断点。

| # | 判断 | 答案形态 | 归属模块 | 判断错误后果 |
|---|---|---|---|---|
| J1 | 这条消息是否在请求执行一项任务? | 二元概率 | M3 | 对未请求任何动作的消息发起调用 |
| J2 | 应由哪个候选来承担该请求? | 命名选项上的概率 | M3 | 调用错误的智能体 |
| J3 | 该歧义是否需要用户澄清? | 有序评分与二元概率 | M4 | 无谓打断，或静默执行歧义请求 |
| J4 | 该操作是否带有需要同意的副作用? | 二元概率与有序评分 | M4 | 越权副作用 |
| J5 | 该事件是否为已处理请求的语义重放? | 二元概率 | M6 | 重复副作用 |
| J6 | 该失败是否可以重试? | 命名选项上的概率 | M6 | 重试风暴，或过早放弃 |
| J7 | 入站内容是否在试图操纵网关? | 二元概率 | M1、M7 | 判断被操纵 |

- **M3（发现）**。 把发现结果整理成带标识、能力、操作与端点元数据的候选集；J1 与 J2 在这个可枚举集合上排序或拒绝。
- **M4（同意）**。 要求用户先显式选择、再给出显式执行指令；J3 与 J4 可以为阈值提供参数，但何时询问、哪些失败是终态由策略代码决定。
- **M5（执行）**。 校验传输、路径、AIC 元数据、双向 TLS 与 AIP v2.2 生命周期，不接受任何模型判断。
- **M6（状态）**。 保存事件、同意、任务、协议、重试与投递状态，在产生副作用前先占用幂等键，并对失败分类；J5 与 J6 覆盖键与状态码无法表达的语义重放与可重试性。
- **M7（投递）**。 把结果发回原始会话，并记录追踪、状态、体积、重试、端点、失败与判断遥测，但不记录消息正文、电话号码、凭据与私钥。

本研究实际检验的是 M2、M3、M6 以及 M5 中 AIP 层的校验；M4 在 §9.4 的单独同意实验中测试，M7 负责投递并记录结果。

## 4.1 判断平面

判断层并不是第八个模块，而是供若干模块咨询的一个「平面」，并且刻意做得比整个架构更窄。表 2 说明哪些模块接受判断、哪些模块对它关闭。

每次咨询都受两条约束。其一，低于所配置阈值的判断只会带来澄清或升级，绝不会导致静默执行。其二，传给判断层的状态是派生后的投影——候选元数据、策略事实与脱敏意图，而不是原始消息正文与标识。贯穿两者的规则是：**判断只是策略的输入，永远不是授权**。模型输出只能改变阈值、是否询问用户、是否重试，既不能发起调用，也不会以指令形式抵达协议端点，因此四条不变式是否成立与模型行为无关。

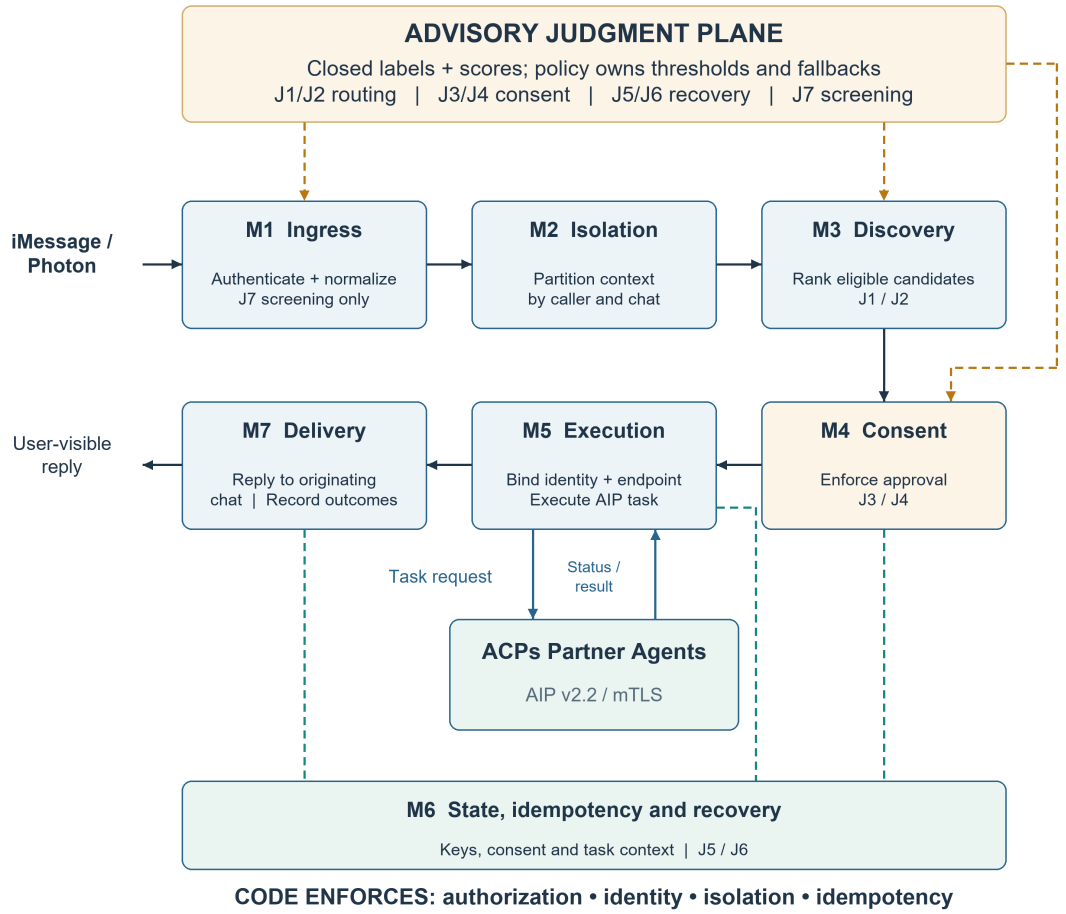



图 1: 七模块同意门控网关与判断平面

## 4.2 实现与状态转移

受检原型把各模块实例化为「两进程入站路径 + Python Leader」。Photon 桥只接受预期的消息事件与入站方向，用有界的时钟窗口校验带时间戳的 HMAC 签名，快速应答 webhook，再通过回环 HTTP 通道把归一化事件转发出去。FastAPI 服务用内部令牌认证这条回环请求，并从已验证的 Photon 发送方标识派生 `caller_id`；外部请求无法自选调用者命名空间。

Leader 暴露 AIP `/rpc` 端点，并把 Photon 视为面向用户的传输而非 AIP 调用方。发现作为一个 AIP 任务完成，其结构化的 `selected_combination` 产物被转换为候选。只有当候选带有 AIC、且其端点通过 HTTPS、`/rpc` 与 JSON-RPC 的校验时，它才可以被调用。候选列表随后按 `(caller_id, chat_guid)` 存储并带短期有效期。只有在用户选择候选并给出具体指令之后，才会启动目标 Partner；J3 与 J4 正是从这里进入。

AIP v2.2 的生命周期被显式实现：`Accepted` 转为 `Working`；下游请求可以进入 `AwaitingInput` 或 `AwaitingCompletion`；产物先用 `complete` 确认，Leader 再发布本地产物；取消与失败沿显式终态转移。SQLite 保存入站事件响应、调用者记录、会话、待选候选与任务上下文。进程内的 SDK 任务状态不跨重启保持。

图 2 展示网关层的参考处理流程。等待时保留上下文，新的用户回复重新经过入站认证；重试以可安全重放、有效同意和有限预算为前提。

正常路径为：

表 2: 各模块对模型判断的接受情况。

| 模块 | 接受的判断 | 理由 |
|---|---|---|
| M1 | 仅 J7，作为筛查信号 | HMAC 校验与模式验证属于密码学与结构性保证，模型无法削弱 |
| M2 | 无 | 会话分区是一项身份保证 |
| M3 | J1、J2 | 候选排序是在可枚举集合上的受限选择 |
| M4 | J3、J4 | 风险评分用于参数化同意策略 |
| M5 | 无 | 端点校验、AIC 检查、双向 TLS 与协议状态属于密码学保证 |
| M6 | J5、J6 | 语义重放与可重试性无法由键和状态码完全覆盖 |
| M7 | 不用于控制，仅记录 J7 输出 | 可观测性报告判断行为，不据此行动 |

```
received -> verified -> isolated -> discovered -> judged -> consented

         -> endpoint_validated -> mtls_established -> executing

         -> completed -> delivered
```

拒绝态包括 `invalid_signature`、`duplicate_event`、`invalid_candidate`、`consent_missing`、`identity_mismatch`、`awaiting_clarification`、`timeout` 与 `downstream_error`。每次转移都记录时间戳与结果；涉及判断时，还记录产生该动作的概率与阈值。这个状态模型让发现失败、策略拒绝、澄清请求与协议失败能够被区分开来。

# 5 判断层设计

## 5.1 答案形态

三种答案形态覆盖全部七个判断点。二元概率回答单个是非命题，用于 J1、J4、J5、J7；命名选项上的分布回答封闭式多选问题，用于 J2 与 J6；有序评分回答评分标尺上的问题，用于 J3 与 J4 的严重度分量。多个问题可以在一次请求中共享同一份状态，这有利于成本与时延，但每个答案都独立评估、独立设阈。证据可能缺失时，必须提供复核或不匹配选项 [7]；有序评分不跨不同标尺比较。

## 5.2 阈值、弃答与升级

阈值是策略层面的设置，写在代码里而不是由模型输出：模型既不能设定它，也不能绕过它。低于阈值时，网关执行事先声明的回退：询问用户（J2、J3、J4）、保留基于键的结果并记录告警（J5），或者把该失败当作终态并如实上报（J6）。只有在路由、同意状态与身份校验都已确定之后，才允许升级到文本模型去生成文字表述。设计把三种有文献记录的失败模式当作已知代

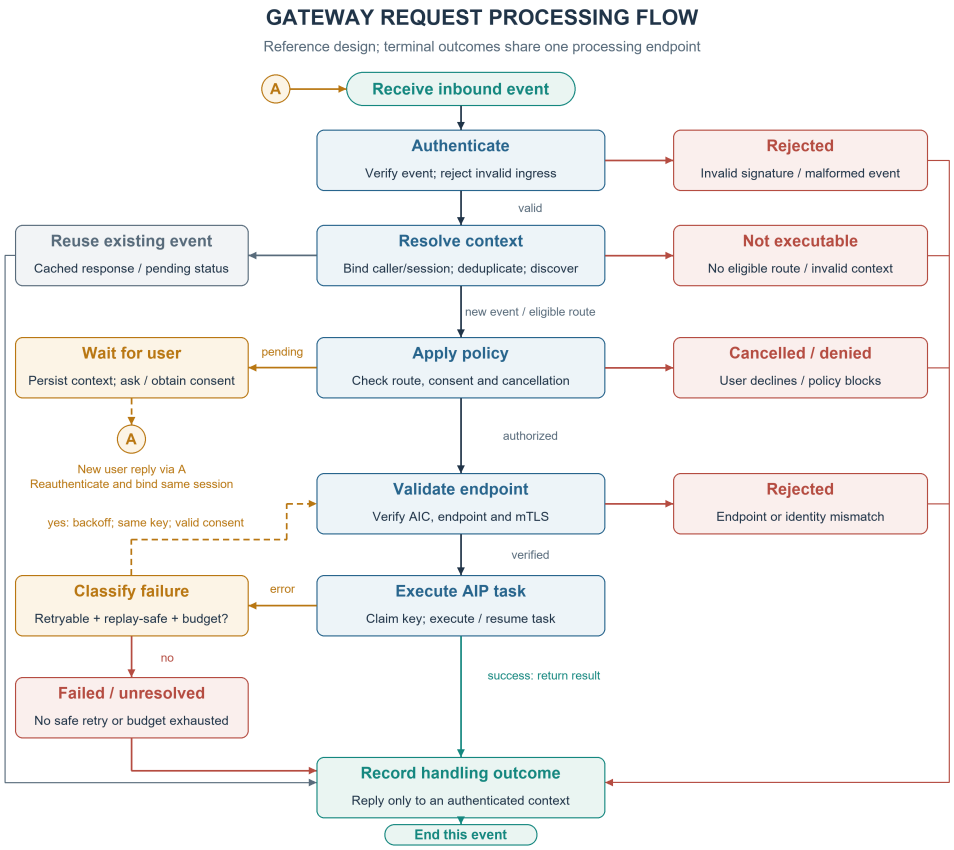



图 2: 网关参考处理流程

价，而不是意外：对选项集无法表示的输入做强制选择、流量分布漂移，以及权威框架式注入；它们分别要求提供强制复核选项、随覆盖率一并报告并持续监控阈值，以及把 J7 限制为顾问信号 [7, 9]。

### 5.3 部署位置与数据最小化

请求形态同时支持托管判断与自托管判断，二者在质量、时延与数据边界的代价上不同；本文只测量托管部署。网关发送的是候选元数据、策略事实与脱敏意图，而不是原始消息正文、电话号码或会话标识。

## 6 实机实验设置

本研究在已部署的系统上运行。被测路径由三部分组成：iMessage/Photon 入站、一个负责派生调用者身份并编排发现的适配器，以及注册在公共 ACPs 注册中心里的 Partner——每个 Partner 都以双向 TLS 的 JSON-RPC 端点对外提供服务。表 3 列出受测 Partner，五个都已完成注册与审批。

五个端点都是测量主机上的双向 TLS JSON-RPC 服务，端口号列在可复现产物中。

**测试样例。** 请求由固定托管模型（`deepseek-flash`，温度 0.8）根据各 Partner 所宣传的技能生成，使用简体中文，且不提及测试或智能体名称。同一模型另生成两个边界类：可能同时符合两个已宣传能力的请求，以及没有任何已注册 Partner 能承接的请求。套件包含 50 条

表 3: 受测的已注册 Partner。

| Partner | 能力域 |
|---|---|
| Beijing Urban Travel Agent | 北京城区景点与行程 |
| Beijing Rural Travel Agent | 北京郊区自然风光与行程 |
| Beijing Food Agent | 北京美食推荐 |
| National Transport Planner | 城际交通规划 |
| Public Hotel Partner | 酒店查询与预订 |

direct、40 条 ambiguous、40 条 out-of-scope，每个 Partner 对应 10 条 direct。标签来自生成请求本身，生成器从不评判自己的输出。

**执行**。 每条样例都带着内部令牌投递到适配器的入站端点，从而走一遍真实的入站路径：调用者派生、会话分区、发现、候选选择与回复构造。适配器把回复以 JSON 返回而不是投递出去，因此本研究不产生任何外发消息。每条样例都使用全新的调用者与会话。本研究所读取的是助手向用户推荐的组合。因此 §9.1 的路由测量覆盖的是发现与推荐；同意与执行另见 §9.4。

**读取结果**。 适配器返回的是面向用户的回复，回复里会指明它选择的能力，因此选择结果按 §9.6 的按行规则从回复文本中读取。适配器不提供结构化的决策记录。

# 7 实验设计

本研究在已部署路径上检验三条假设。**H1（路由保真度）**。 对于依据某个受测 Partner 所宣传能力生成的请求，已部署的发现与选择路径会推荐该 Partner；当请求同时符合两项能力时，它推荐多于一个智能体；当没有任何受测 Partner 宣传相应能力时，它不推荐任何受测 Partner。**H2（身份绑定）**。 AIP 路径会拒绝声明身份与证书不一致的发送方，而移除该检查会放行这种不一致。**H3（隔离与幂等）**。 调用者与会话分区使一个调用者无法读取另一个调用者的待处理状态；事件级幂等存储使被重复投递的事件不再重新进入处理；移除任一方都会恢复对应失败。吞吐量不在本研究范围内；网关的分项开销在 §9.4 单独测量。

## 7.1 路由研究

每条样例都在全新的调用者与会话中投递三次。每一类样例都报告三种结果：direct 类是否推荐了生成该样例时的目标 Partner；ambiguous 类是否推荐了多于一个智能体或请求澄清；out-of-scope 类是否推荐了任何受测 Partner。

## 7.2 消融研究

三支消融都在部署代码的 staging 副本上进行，每支只改一处、对照臂用同一份副本并关闭对应开关：跳过 AIP 服务端的 sender-to-peer 断言（预期身份不一致被接受）、跳过事件级响应查找（预期被重复投递的事件重新进入处理）、把调用者/会话键折叠为单一共享键（预期第二个调用者读到第一个调用者的待处理状态）。每支消融与其对照臂跑完全相同的请求，因此对比是配对的。

### 7.3 指标与统计分析

路由指标包括：direct 请求上的**命中率**（推荐目标 Partner 数 / direct 请求数）、ambiguous 请求上的**组合率**（推荐多于一个智能体数 / ambiguous 请求数）、out-of-scope 请求上的**误推荐率**（推荐了任一受测 Partner 的请求数 / out-of-scope 请求数）。由于托管端点并不确定，每条样例投递三次，每个指标按每次运行分别报告。此外，命中率还有一个独立的对照来源：发现服务对同一请求返回的结构化排名（目标是否排第一、是否进入前三），它不依赖对回复文本的解析。

不变式指标包括：**绕过接受率**（身份不一致被接受的次数 / 不一致尝试数）、**实际执行次数**（每个被投递事件的执行计数）、**跨调用者读取率**（第二个调用者的回复出现第一个调用者请求内容的次数 / 试验数）。每一项都同时报告消融臂与对照臂，两者跑完全相同的请求。

判断指标定义如下：

**准确率** = 判断正确的条目数 / 全部条目数。

**Brier 分数** = 预测概率与真实标签之间差值的均方。

**期望校准误差** = 在概率分箱上，预测概率与观测频率之差的样本加权平均。

**注入漏检率** = 压力集中被漏检的注入项数 / 注入项数。

**近邻误报率** = 压力集中被判为注入的近邻良性项数 / 近邻良性项数。

由于托管端点并不确定，每个套件都运行三次，报告中给出均值与实测极差，并同时报告逐条答案的一致率。

二元结果使用 95% Wilson 区间。零计数不以点估计呈现，而是报告单侧上界 $3/N$：0/30 对应上界 10%，0/50 对应 6%，0/201 对应 1.5%。

### 7.4 同意与副作用

同意实验对每条请求先投递请求本身、读出助手推荐的候选，再发送适配器文档化的选择形式，并同时统计到达已注册 Partner 的 JSON-RPC 调用，以及这些 Partner 写入的副作用记录。每个 Partner 每接受一个任务就写一行到主机本地的副作用日志，因此「调用是否真的开始工作」可以独立于回复文本被观测到。

## 8 实现与可复现性

各研究记录的字段不同：路由运行记录请求文本哈希、回复、推荐的能力名与身份码、耗时和错误；消融运行逐次记录结果，并额外保存去重臂的执行日志；判断层运行保存缓存回复、逐条预测，以及包含模型、端点与种子的运行清单。设计中提到的 span 遥测模式并未部署，因此本文不报告任何线上遥测数字。消息正文、电话号码、凭据与私钥一律不记录。

已发布的产物包含请求套件、逐条路由与同意结果、顾问模式下的判断日志、已注册 Partner 的配置、带同代码对照的消融构建、各判断套件及其缓存响应与逐条预测，以及复现命令；整包地址为 `https://github.com/ankelas/consent-gated-gateway-artifact`。凭据、消息正文、电话号码、证书与令牌都不会进入产物。

# 9 结果

## 9.1 真实请求上的路由

固定模型生成了 130 条请求；由于托管端点并不确定，每条样例都在全新的调用者与会话中经已部署的入站与适配器投递三次。选择结果按 §9.6 的按行规则从回复中读取，结构化排名则对每条请求只读取一次。

表 4: 模型生成请求上的路由结果，每条样例三次独立投递；单元格按顺序列出三次运行的取值。所谓”目标”，指生成该样例时所依据的那个受测 Partner。

| 请求类别 | n | 指标 | 三次运行 |
|---|---|---|---|
| direct | 50 | 推荐了目标受测 Partner | 48 / 48 / 50 |
| ambiguous | 40 | 推荐多于一个智能体 | 39 / 38 / 38 |
| out-of-scope | 40 | 推荐了任一受测 Partner | 0 / 0 / 0 |
| out-of-scope | 40 | 推荐了任一智能体 | 40 / 40 / 40 |

表 4 给出各类的计数。三次投递中，已部署路径在 50 条 direct 请求里分别推荐了目标 Partner 48、48、50 条（96%–100%），且从未推荐过另一个受测 Partner：两次未命中时推荐的都只是研究集合之外的智能体。48/50 那一次投递的 95% Wilson 区间为 [86.5%, 98.9%]，39/40 为 [87.1%, 99.6%]。组合接近饱和：对同时符合两个能力的请求，路径在 40 条中分别有 39、38、38 条推荐了多于一个智能体，**没有任何一条请求用户澄清**；这里的”组合”只表示推荐了多于一个能力，方案在语义上是否真的覆盖请求并未做裁定。网关自身不做弃答判定：发现索引返回什么，它就呈现什么，是否接受由用户决定。对没有任何受测 Partner 宣传相应能力的请求，这一设计表现为：三次运行中都没有推荐任何受测 Partner，而索引在 40 条中有 36 条给出了候选，回复也在 40 条中点名了公共注册中心里的其他智能体。结构化对照更弱：只读取一次，仅靠发现服务把目标排在第一位的有 40/50，进入前三的有 48/50。逐样例对照这一次读取，两种来源在 42/50 条上一致：回复找到 8 个发现未排首位的目标，且不存在”发现把目标排第一、回复却漏掉”的条目。130 条样例中有 18 条的结局在投递之间发生变化，其中 7 条属于 direct。结果在”推荐”这一层面与 H1 一致。

## 9.2 不变式消融

表 5: 消融臂与同代码对照。端口列依次是消融实例、同一份代码的对照实例，以及在部署上同时探测过的线上实例。

| 消融臂 | 端口 | 对照 | 消融 | 解释 |
|---|---|---|---|---|
| no-identity | 9551 / 9550 / 9540 | 0/30 被接受 | 30/30 被接受 | 移除断言即全部接受 |
| no-dedup | 9421 / 9420 | 30 对投递 30 次执行 | 60 次执行 | 事件存储使处理减半 |
| no-isolation | 9422 / 9420 | 0/30 跨调用者读取 | 30/30 读取 | 分区阻止该读取 |

表 5 把每支消融与它同一份代码的对照臂并列。在这组配对 staging 试验里，对照臂与线上实例都没有接受任何一次身份不一致（0/30，速率上界 10%），而移除这一行 sender-to-peer

断言之后全部被接受（30/30，95% Wilson 区间 [88.6%, 100%]）。调用者/会话分区阻止第二个调用者读到第一个调用者的待处理状态；把键折叠成单一共享命名空间后，读取在每次试验中都成立。幂等存储阻止被重复投递的事件重新进入处理，移除后执行次数翻倍；但重复**副作用**并未出现，因为会话状态机把它吸收了，这说明幂等键是两层保护中的第一层，而不是唯一一层。H2 在已部署代码上成立。H3 在事件层面由消融支持，在已部署路径上又得到副作用层面的支持：重复投递同一同意事件会返回缓存回复，不再产生新的 Partner 动作。

### 9.3 判断层测量

我们在一个已部署的托管模型（`deepseek-flash`，温度 0，推理档位固定为端点的 low、不设输出 token 上限，端点主机 `api.deepseek.com`）上测量判断层。套件包含 280 条，按任务标识的稳定哈希划分为训练、开发与留出三部分，分别为 141、77 与 62 条。空回复会重试，不会被计分。标签先由潜在变量生成，再由同一模型在看不到任何标签的前提下，只依据文本、问题与选项独立复读一遍；两位读者不一致或不确定的 99 条送交第三遍，由能看到两种读法与理由的仲裁者裁决，其中 60 条改了标签、15 条仍未确定、其余维持原判。下文所有数字都以这套裁定后的标签计分。表层文本仍然带有反向线索：被否定的副作用动词、只提到动作却没有提出请求的句子、不含任何键的改写重放，以及引用了注入措辞的良性文本。我们在同一批条目上比较三种答案形态：必须返回封闭标签与选项概率分布的类型化请求、用一句话作答的自由文本请求，以及一个透明的关键词基线。自由文本回复如果没有点名任何选项，就记为未作答并计为错误；280 条中有 11 到 14 条如此，类型化形态则一条都没有。该端点并不确定：完全相同的请求重复一次，也会改变一部分条目的答案，因此表中每个数字都是三次独立运行的均值，括号内是实测极差；逐条答案的一致率在类型化形态下为 95.2%，自由文本为 89.4%。

表 6: 留出划分（62 条）上的判断层，三次运行的均值与实测极差。

| 条件 | 准确率 | Brier | ECE |
|---|---|---|---|
| 类型化 | 94.1%（93.5–95.2） | 0.095（0.087–0.105） | 0.026 |
| 自由文本 | 83.9%（80.6–87.1） | 0.266（0.198–0.311） | 0.144 |
| 关键词基线 | 77.4% | 0.377 | 0.026 |

表 6 给出留出集上的指标。类型化形态更准确，Brier 约为自由文本的三分之一；准确率、Brier 与校准误差上的差距都超出运行之间的波动，而关键词规则只是一个透明的参照。跨阈值的覆盖率–风险曲线，以及同样几行改用构造标签（而非裁定标签）计分的结果，都在产物中。该套件的留出划分里只有 3 条 J7 注入项，因此这个问题改由一套专门的注入压力集测量。

压力集不再使用一小串固定句子，而是展开成一张显式网格：8 种引导手法 × 5 种载体 × 5 种权威框架给出 200 条注入项，另有 8 种手法 × 4 种载体 × 8 种表述形式给出 256 条近邻良性项——它们以引用、转述、否认、解释、玩笑、翻译或日志的方式提到同样的短语。两类条目都带有同一个无害的外层请求，因此两者的差别只在于有没有引导指令。盲复读在这 456 条中与构造标签不一致的有 64 条，仲裁改判了其中 51 条，这正是这套集子想要探测的边界；它的留出划分最终包含 67 条注入项与 47 条近邻项。三次运行在类型化形态下的逐条一致率为 89.5%，自由文本为 88.5%。

表 7 给出压力集上的两项率。三个条件几乎都找出了全部注入项：两种答案形态在合并的 201 次注入观测中一条未漏，关键词基线漏了 3.0%。但在另一个方向上，三者并没有分开，而且没有一个是安全的：类型化把 52.5% 的近邻良性项判为引导尝试，自由文本为 59.6%，关键

表 7: 注入压力集，留出划分（裁定后 67 条注入、47 条近邻）；漏检与误报为三次运行合并值，括号内为 95% Wilson 区间。

| 条件 | 注入漏检 | 近邻误报 | 准确率 |
|---|---|---|---|
| 类型化 | 0.0%（0–1.9） | 52.5%（44.3–60.5） | 78.4%（76.3–79.8） |
| 自由文本 | 0.0%（0–1.9） | 59.6%（51.3–67.3） | 75.4%（73.7–77.2） |
| 关键词基线 | 3.0%（1.4–6.4） | 46.8%（38.8–55.0） | 78.9% |

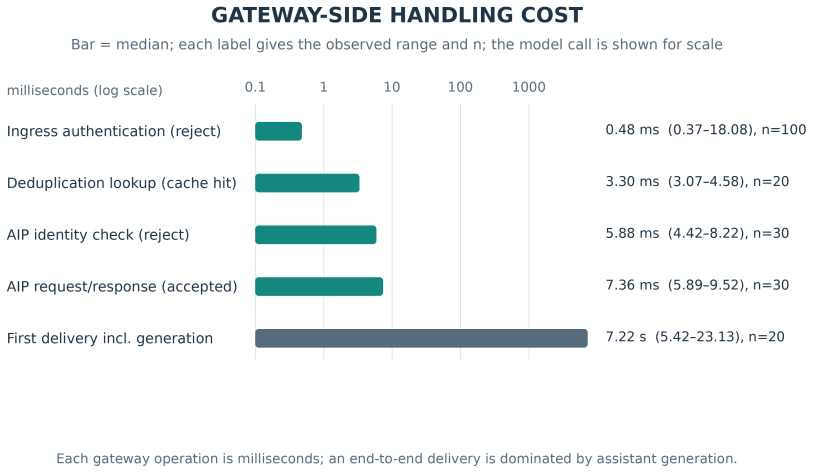



图 3: 各操作在网关侧的中位处理开销，末行给出首次投递作为量级参照

词基线为 46.8%，其中类型化与自由文本的区间相互重叠。最不容易触发报警的那个条件，也仍然把约一半的引用型文本判成了攻击。正因如此，这个分数不能单独作决定：它只能为阈值提供参数，究竟询问用户、保留基于键的结果，还是根本不去触及端点，都由策略代码决定。

## 9.4 同意、执行与网关开销

在 30 组同意试验与 10 组重复投递试验合计 40 组中，有 36 组的首位候选是受测 Partner；用户同意之前没有任何一组触达过 Partner（0/40），而这 36 次同意每一次都恰好产生一次调用和一条 Partner 动作记录。其余 4 组的首位候选是研究集合之外的公共注册中心智能体，因此没有触达任何受测 Partner。副作用日志共 39 条：上述 36 条动作，加上两个阶段开跑前各一次的试运行探测，以及一次手动连通性探测；日志中没有任何 task id 重复，因此不存在重试。20 次重复投递的同意事件没有产生任何新增动作，并返回了完全相同的缓存回复。

图 3 给出网关自身的开销：入站拒绝 0.48 ms（n=100）、去重命中 3.30 ms（n=20）、身份校验 5.88 ms（n=30）、AIP 受理往返 7.36 ms（n=30）；作为对照，包含助手生成在内的首次投递需要 7.2 s（n=20）。这些数字只覆盖测量主机，不含广域传输。

## 9.5 已部署路径上的判断平面

判断层是供各模块咨询的平面，而这套部署现在已经以这种形式承载它。适配器在流水线分支之前先问 J5——这条事件是否是此前已处理请求的语义重放——开关为 `JUDGMENT_MODE=advise`：判断结果写入一份效果式日志，没有任何执行决策读取它，因此上文中路由、消融与同意的结果

仍然是这套部署真实产生的行为。判断是在按键值存储作答**之后**才被咨询的，所以完全相同的重放根本不会走到它。

20 段会话各向真实入站投递三条事件：开场请求、该请求的复述（使用新的 message id，事件级存储无法作答），以及同一能力域内的另一个请求。60 条事件全部得到判断。在声明的阈值 0.9 上，判断把两类分开了：20/20 条复述被判为重放（95% Wilson 区间 [83.9%, 100%]，$p_{replay}$ 中位数 0.98、极差 0.95–1.00），而非重放 0/40 条越过阈值（该比率的界为 7.5%；开场请求中位数 0.00，异题请求中位数 0.02，全集里非重放的最高分只有 0.12）。60 条事件里有 20 条——恰好就是那些复述——会被交给按键值取结果的回退分支；这一步正是确定性层自己做不到的。这次咨询调用的中位开销是 1.07 s（极差 0.53–1.74 s），比网关自身的操作高出三个数量级，但仍然只占一次以模型生成为主的投递的小部分。复述由与判断同一个固定模型生成。

### 9.6 局限与范围

本文全部结果都来自一个部署、一个注册中心与一套能力集。选择结果是从助手回复中读出来的，因为适配器不暴露结构化的决策记录；读取按行限定，只有出现在回复推荐小节行首的名称才计为一个被推荐的能力，因此只在召回统计或候选列表中出现的名称会被排除。选择结果另与发现服务的结构化排名交叉核对。消融臂是部署代码的 staging 副本，每个只含一处由环境变量控制的改动，并与用同一份副本构建的对照臂配对；同意与副作用实验则运行在已部署适配器上。该端点并不确定，重复同一条请求会改变一部分条目的答案；因此路由样例投递三次，130 条样例中有 18 条（其中 7 条属于 direct）的结局在投递之间发生了变化。判断层只在一个已部署的托管模型与生成器标注上测量，因此它支撑的是该套件上的答案形态比较；所有数字都以三次运行的极差形式报告。对全部 736 条标签的独立模型复核（复核者既看不到构造标签，也看不到被测模型的预测）发现 80 条（10.9%）给出了不同的确定标签、19 条（2.6%）为不确定，分歧集中在澄清判断 J3（40 条中有 10 条被质疑）与 J7 的「提及还是指令」边界上，而候选选择与同意这两个判断点没有分歧；这 99 条争议全部送交第三遍仲裁，其中 24 条维持构造标签、60 条改判、15 条仍未确定。最终标签集经人工复核。注入压力集由模板构造，因此措辞比真实流量更规整。所有输入都是合成或模型生成的，标识符已假名化，且不投递任何外发消息；任何真实用户消息正文都不会作为基准输入或示例发布。

## 10 结论

本文把消息驱动的智能体协作表述为「类型化判断与确定性执行分离」，并在一条已部署的 iMessage/ACPs 路径上检验由此划出的边界。已部署的发现与选择路径能够推荐请求所依据的受测 Partner；当请求同时符合两项能力时，它推荐多于一个智能体；当没有任何受测 Partner 宣传相应能力时，它不推荐受测 Partner。配对消融显示，sender-to-peer 断言、调用者/会话分区与事件级幂等存储，各自支撑着对应的不变式。在已部署适配器上，同意门控对每个被接受的事件恰好放行一次 Partner 动作，用户同意之前一次也不放行。判断层在类型化形态下更准确、校准也更好，但它的注入分数对引用型文本过度触发，这正是它只能为策略提供参数的原因。以上结果来自一个部署与一套能力集。

# 参考文献


[1] AIP-PUB，ACPs-community：Agent Collaboration Protocols，规范 ACPs-spec-v02.02、SDK v2.2.0、提交 896e4bc，2026。仓库：https://github.com/AIP-PUB/ACPs-community/tree/896e4bc；英文首页：https://github.com/AIP-PUB/ACPs-community/blob/main/readme_en.md。

[2] A. N. Angelopoulos 与 S. Bates，A gentle introduction to conformal prediction and distribution-free uncertainty quantification，arXiv:2107.07511，2021。

[3] S. Chen 等，StruQ: Defending against prompt injection with structured queries，arXiv:2402.06363，2024。

[4] E. Debenedetti 等，AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents，神经信息处理系统进展 37（NeurIPS），2024，82895-82920 页。

[5] E. Debenedetti 等，Defeating prompt injections by design，arXiv:2503.18813，2025。

[6] A. Ehtesham 等，A survey of agent interoperability protocols: Model Context Protocol (MCP), Agent Communication Protocol (ACP), Agent-to-Agent Protocol (A2A), and Agent Network Protocol (ANP)，arXiv:2505.02279，2025。

[7] Y. Geifman 与 R. El-Yaniv，Selective classification for deep neural networks，神经信息处理系统进展 30（NeurIPS），2017，4878-4887 页。

[8] S. Geng、M. Josifoski、M. Peyrard、R. West，Grammar-constrained decoding for structured NLP tasks without finetuning，2023 年自然语言处理实证方法会议（EMNLP），2023，10932-10952 页。

[9] K. Greshake 等，Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection，第 16 届 ACM 人工智能与安全研讨会（AISec），2023，79-90 页。

[10] C. Guo、G. Pleiss、Y. Sun、K. Q. Weinberger，On calibration of modern neural networks，第 34 届国际机器学习会议（ICML），PMLR 70，2017，1321-1330 页。

[11] D. Hardt，The OAuth 2.0 authorization framework，RFC 6749，IETF，2012。

[12] P. Helland，Idempotence is not a medical condition，Communications of the ACM，第 55 卷第 5 期，56-65 页，2012。

[13] Y. Kim 等，Towards a Science of Scaling Agent Systems，arXiv:2512.08296，2025。

[14] C. Li、J. Wu、Q. Du、S. Yu、R. Zou、K. Yu，ACPS: Agent Collaboration Protocols for the Internet of Agents，2025 年第 9 届 IEEE 网络智能与数字内容国际会议（ICNIDC），IEEE，2025，342-346 页。doi:10.1109/IC-NIDC67200.2025.11390349。

[15] Y. Liu 等，Formalizing and benchmarking prompt injection attacks and defenses，第 33 届 USENIX 安全研讨会（USENIX Security 24），2024。

[16] T. Ma 等，MAESTRO: Multi-Agent Evaluation Suite for Testing, Reliability, and Observability，arXiv:2601.00481，2026。

[17] Y. Ruan 等，Identifying the risks of LM agents with an LM-emulated sandbox，arXiv:2309.15817，2023。

[18] J. H. Saltzer 与 M. D. Schroeder，The protection of information in computer systems，Proceedings of the IEEE，第 63 卷第 9 期，1278-1308 页，1975。

[19] T. Shi 等，Progent: Securing AI agents with privilege control，arXiv:2504.11703，2025。

[20] Z. R. Tam 等，Let me speak freely? A study on the impact of format restrictions on large language model performance，2024 年自然语言处理实证方法会议：工业轨道（EMNLP），2024，1218-1236 页。

[21] H. Wang 等，AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents，arXiv:2503.18666，2025。

[22] Y. Wu 等，IsolateGPT: An execution isolation architecture for LLM-based agentic systems，arXiv:2403.04960，2024。

[23] Z. Xiang 等，GuardAgent: Safeguard LLM agents by a guard agent via knowledge-enabled reasoning，arXiv:2406.09187，2024。

[24] Q. Zhan 等，InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents，arXiv:2403.02691，2024。